

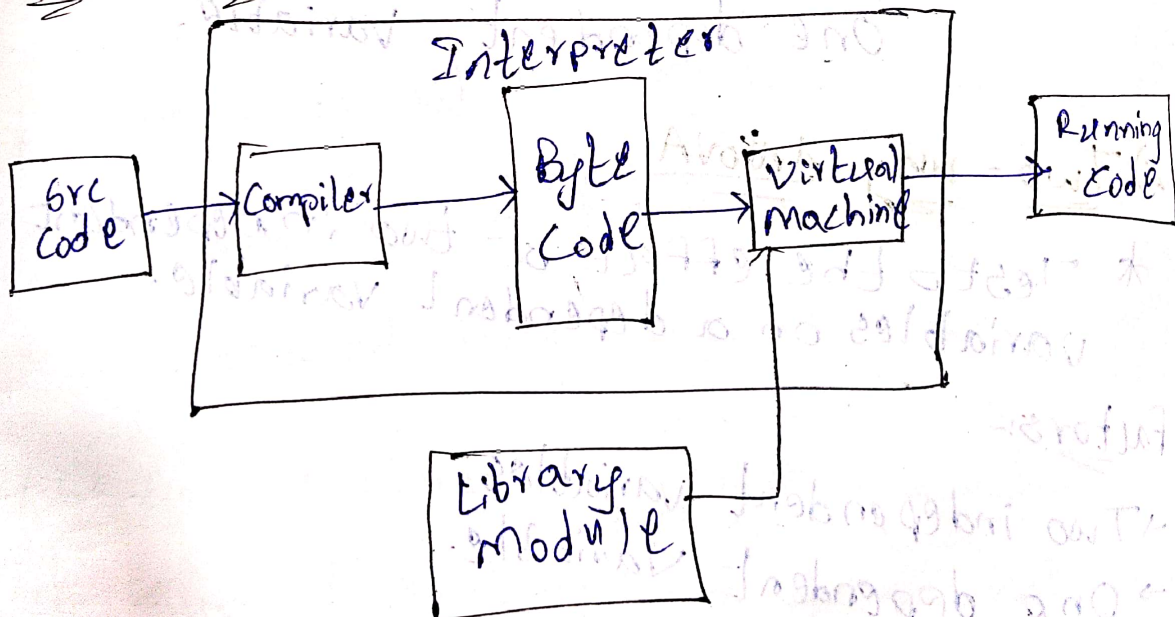
File Extensions in Python

- 1) .py
- 2) .pyc
- 3) .pyd
- 4) .pyo
- 5) .pyw
- 6) .pyz

Applications of Python

- 1) Web & Internet Development, Business Applications.
- 2) Games and 3D Graphics
- 3) Network programming
- 4) Database access.
- 5) Database access.
- 6) Network Programming

How python Run's



DATA TYPES

- 1) Lists
- 2) Dictionary
- 3) Tuple
- 4) Set

STRING METHODS

- title()
- upper()
- lower()
- isdigit()
- isupper()

Lists

Syntax →

```
list = [value1, value2, value3]
```

Ex:- `List = ["apple", "banana", "cherry"]`

`print(List)`

O/p:- `['apple', 'banana', 'cherry']`

Ex:- `mixed-list = [1, "hello", 3.14, True]`
`print(mixed-list)`

O/p:- `[1, "hello", 3.14, True]`

~~Array~~

Reshape NumPy Array

12/09/25

* Changing the shape of the Array.

→ Reshaping 1D to 2D array.

Ex:- `Array : [1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16]`

`n = array.size`

`print(n)`

⇒ 16

Ex:- `N = 4`

`m = n/N`

`print(N)` = 4

`print(M)` = 4

`reshaped_1 = array.reshape(N, m)`
`print(reshaped_1)`

output:-

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

reshaped 2 = np.reshape(array, (2, 8))
print(reshaped 2)

O/p:-

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

~~10 to 20~~

Why Python

- * It is simple
- * Versatile & Flexible
- * Support AI
- * Expandable
- * Mature Package Libraries
- * Beginner Friendly.

Web Frameworks

- Flask
- Django
- Pyramid
- Web2Py
- CherryPy
- Tornado

* A web framework is a software tool or platform that provides a standard way to build and deploy web applications.

Single-line Comments

(#) $\Rightarrow$ hash character is used for single line comments.

Multi-line Comments

(""") $\Rightarrow$ used for multi-line comments.

* Numpy is a fundamental package for scientific computing in python.

Ex:- import numpy as np
a = np.array([1, 2, 3, 4])
print(a)

O/P:- [1, 2, 3, 4] $\Rightarrow$ Single-Dimensional Array

Ex:- import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a).

* concatenate function is used to -
add strings.

ex:- import numpy as np
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
arr2 = np.array([[7, 8, 9], [10, 11, 12]])
result = np.concatenate((arr1, arr2))
print(result)

Time Series Data with Numpy | 19/09/25

* Data is collected over time in consistent intervals.

Ex:- hours, days, years.

* Used in many analysis, especially in businesses.

Ex:- → Data forecasting

→ Anomaly detection

→ Trend analysis.

Types of Dataset

Pandas Series

Ex:- Import pandas as pd

```
data = [10, 20, 30, 40]
```

```
s = pd.Series(data)
```

```
print(s)
```

o/p:-

0 10

1 20

2 30

3 40

* Database == Dataset == DataFrame.

Creating a dataframe from dict to nd array

Ex:- import pandas as pd

```
data = {'Name': ['Tom', 'nick', 'krish', 'jack'],
```

```
       'Age': [20, 21, 19, 18]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output:-

	Name	Age
0	Tom	20
1	Nick	21
2	Krish	19
3	Jack	18

Aggregation

* Grouping or combining the data is called aggregation.

* Combination of all the relations & entities into an ER diagram is called aggregation in database.

* Where data is grouped from one or more variables.

* Aggregation serves the purpose of simplifying complex data, reducing redundancy, & extracting key information in any context.

Time-Series
Summarizing data over some
time-series framing.

* Aggregating data hierarchically allow
for the better analysis of the data

→ Rollup
→ Drill down ⇒ Expansion of rollup.

* Pivot ⇒ Reverse of 1 cube.

* Aggregation is the form of data
reduction, By this the dataset become
smaller & more manageable.

Aggregation in Pandas

* This is a process of applying a
function ^{Set} to a dataset.

→ mean()

Ex: average-age = df['Age'].mean()

Print (average-age)

O/p: 30.0

→ median()

median-salary = df['Salary'].median()

print (median-salary)

O/p: 50000

→ count(), max(), std() & var()

agg()

```
Summary_stats = df['salary'].agg  
(['mean', 'median', 'std'])
```

```
print(summary_stats)
```

Concatenation

* Combining two strings, Lists, DataFrames

Time-series Analysis

* This is a sequence of data points collected or recorded at successive points in time.

Ex:- Daily stock prices of a company.

Cross-sectional Data

* At a single point in time across multiple subjects or entities.

Ex:- Income levels of 1000 households in 2020.

Components

1) Trend

2) seasonality $\Rightarrow$ Regular, repeating data patterns in the regular intervals of time.

Noise

* Random variation or unpredictable fluctuations that cannot be explained by theory.

Autocorrelation

* Measures the correlation of a time series with a lagged version of itself.

Auto Correlation Function

- X-axis: Lag (e.g. 1, 2, 3)
- Y-axis: Lag ranges from -1 to +1.

Plotting

Types of Distribution

- 1) Pairwise Data
- 2) Statistical Distribution.
- 3) Gridded Data.
- 4) 3D and volumetric data.

Libraries

→ Matplotlib

→ Seaborn

→ Pandas

→ Plotly

→ Altair

→ HoloViews

→ ggplot

→ Geopandas

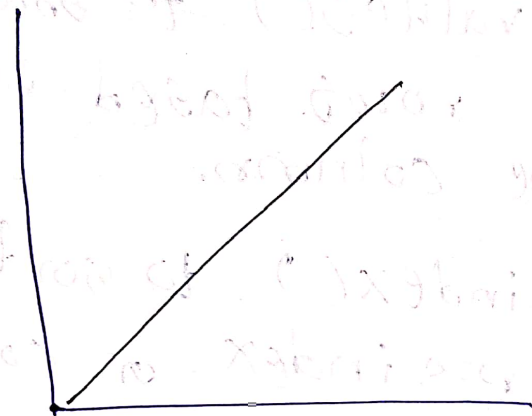
→ Mayavi

16/10/25

Line Plotting

17/10/25

* Represents the data x & y
↓
relation b/w



Numerical Data

- Line plot matplotlib's plot.
- Time-Series Data
- we use Matplotlib's or Seaborn plot

Scatter Plotting

* Represented by dots.

* Syntax →

(x-axis = data, y-axis = data, s = Name, c = None,
marker = None, comp = None, wmin = None,
wmax = None).

* Matplotlib library is used.

* Colours are indicated with "Viridis".

low value (near 0) → dark purple.

mid value (around 50) → green.

high value (near 100) → yellow.

* $\alpha \Rightarrow$ adjust the transparency of the data.

* `.sort_values()` to sort the Data Frame's rows based on the values in the column.

* `.sort_index()` to sort a Data Frame by its row index or column labels.

Data Frame

* This is a two-dimensional size-mutable, potentially heterogeneous tabular data structure.

Creation of Data Frame with `pd.DataFrame()`
Ex: `import pandas as pd`

```
df = pd.DataFrame()
```

```
names = ["Sakhar", "Mari", "Manoj", "Raman"]
```

```
Marks = ["95", "75", "98", "87"]
```

```
df["names"] = names
```

```
df["marks"] = Marks
```

```
print(df)
```

O/p:-

	names	marks
0	Sakhar	95
1	Mari	75
2	Manoj	98

Creating Data Frame from dict of ndarray

```
import pandas as pd  
data = {'Name': ['Tom', 'nick', 'krish', 'jack'],  
        'Age': [20, 21, 19, 18]}
```

```
df = pd.DataFrame(data)  
print(df)
```

* We use the `loc` attribute to return the specified rows.
⇒ `print(df.loc[0])`

Data Frame methods

- `head([n])`, `tail([n])` ⇒ First/last n rows.
- `describe()` ⇒ for numeric columns
- `max()`, `min()` ⇒ returns max/min value for all numeric columns
- `mean()`, `median()` ⇒ returns mean/median value for all numeric columns.
- `std()` ⇒ Standard deviation.
- `sample([n])` ⇒ returns a random sample of the data frame.
- `dropna()` ⇒ drop all the records with missing values.

Slicing (Data Frames)

→ `df['salary']` ⇒ This is a series, not a dataframe.

→ `df[['rank', 'salary']]` ⇒ This is a dataframe.

* `df[10:20]` ⇒ This returns the rows from 10 to 19.

Loading files into a Data frame

* import pandas as pd

`df = pd.read_csv('data.csv')`

`print(df)`

↓
reading a csv file.

Aggregation Types

→ Temporal Aggregation

* This summarizes data over specific time intervals.

* This helps in identifying long-term trends & reduces the impact of the noise in the data.

Hierarchical Aggregation

* Data is structured hierarchically.

Ex:- Store, region, & national level data, sales data.

Techniques

- roll-up
- drill-down
- slice and dice
- pivot tables
- grouping.

Data Reduction

* Aggregation is a form of data reduction. By summarizing the data the dataset becomes smaller & more manageable.

* Aggregation function in Pandas:

- sum()
- mean()
- count()
- max()
- min()

Concatenation

Strings

Ex:- str1 = "Hello"

str2 = "World"

result = str1 + " " + str2

Lists

Ex:- list1 = [1, 2, 3]

list2 = [4, 5, 6]

result = list1 + list2

DataFrames

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})

df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

result = pd.concat([df1, df2], axis=0)

group by method

→ By this we can split the data into groups based on some criteria.

→ Calculate statistics to each group.

df_rank = df.groupby(['rank'])

OLP:-

	Phd	Service	Salary
rank: 15	11	9178	
Assoc Prof	5.05	2.21	8132.7
Asst Prof	27.065	21.413	123624.80

df.group by ('rank') ['salary'].mean()
↓

df:

rank	salary
Assoc Prof	91786.230769
Asst Prof	81362.789474
Prof	123621.804348